Document Number:	**p2927r1**
Date:	2024-02-14
Target:	LEWG
Revises:	p2927r0
Reply to:	Arthur O'Dwyer (arthur.j.odwyer@gmail.com), Gor Nishanov (gorn@microsoft.com)

Inspecting exception_ptr

Abstract

Provide facility to observe exceptions stored in `std::exception_ptr` without throwing or catching exceptions.

Introduction

This is a followup to two previous papers in this area:

Date	Link	Title
Feb 7, 2018	https://wg21.link/p0933	Runtime introspection of exception_ptr
Oct 6, 2018	https://wg21.link/p1066	How to catch an exception_ptr without even try-ing

These papers received positive feedback. In 2018 Rapperswil meeting, EWG expressed strong desire in having such facility. This was reaffirmed in 2023 Kona meeting.

This paper brings back exception_ptr inspection facility in a simplified form addressing the earlier feedback.

Proposal at a glance

We introduce a single function `try_cast` that takes `std::exception_ptr` as an argument `e` and returns a pointer to an object referred to by `e`.

```
template <typename T>
const T*
try_cast(const exception_ptr& e) noexcept;
```

Example:

Given the following error classes:

```
struct Foo {
    virtual ~Foo() {}
    int i = 1;
};
struct Bar : Foo, std::logic_error {
    Bar() : std::logic_error("This is Bar exception") {}
    int j = 2;
};
struct Baz : Bar {};
```

The execution of the following program

```
int main() {
    const auto exp = std::make_exception_ptr(Baz());
    if (auto* x = std::try_cast<Baz>(exp))
        printf("got '%s' i: %d j: %d\n", typeid(*x).name(), x->i, x->j);
    if (auto* x = std::try_cast<Bar>(exp))
        printf("got '%s' i: %d j: %d\n", typeid(*x).name(), x->i, x->j);
    if (auto* x = std::try_cast<Foo>(exp))
        printf("got '%s' i: %d\n", typeid(*x).name(), x->i);
}
```

results in this output:

```
got '3Baz' what:'This is Bar exception' i: 1 j: 2
got '3Baz' what:'This is Bar exception' i: 1 j: 2
got '3Baz' i: 1
```

See implementation for GCC and MSVC using available (but undocumented) APIs https://godbolt.org/z/E8n69xKjs.

Simplification post Kona 2023

Previous revision tentatively proposed a complicated signature imitating the syntax of a catch-parameter, as in `old::try_cast<const std::exception&>(p)`. In Kona, we were convinced

to simplify the signature to assume catch-by-const-reference no matter what: `std::try_cast<std::exception>(p)`.

- With the old design, there were two ways to simulate catching by const reference: `old::try_cast<const std::exception&>(p)` and `old::try_cast<std::exception>(p)`. The new syntax removes that needless variability of style.
- Users of the old syntax might think they had to write `old::try_cast<const std::exception&>(p)` in order to catch by reference; that's unnecessarily verbose and hard to read. The new syntax is short and readable.
- The old syntax permitted `old::try_cast<std::exception&>(p)` to catch by non-const reference; this is sometimes legitimate but usually it's just an unnecessary violation of const-correctness. Users might omit the `const` out of inattention or laziness. The new syntax always returns `const*`, privileging the common and const-correct case.
- Users who want to modify the in-flight exception object can still explicitly write `const_cast<std::exception*>(std::try_cast<std::exception>(p))`. The `const_cast` is visible and greppable; this is a good thing.
- The old syntax permitted nonsensical scenarios such as `old::try_cast<int(&)()>(p)`. Such a catch handler is physically impossible to hit, because you can't throw a function; functions are not objects. The new syntax admits only `old::try_cast<int()>(p)`, which is ill-formed by our new Mandates element because `int()` is not an object type. Similarly, we disallow `old::try_cast<int[5]>(p)` via the Mandates element.

P2927R0 proposed that `try_cast` should be able to catch pointer types, just like an ordinary catch clause. That is, not only were you allowed to inspect a thrown `Derived` object with `old::try_cast<const Base&>` (which would return a possibly null `const Base*`), you were also allowed to inspect a thrown `Derived*` object with `old::try_cast<const Base*>` (which would return a possibly null `const Base**`). This turned out to be unimplementable. When a catch-handler catches `Derived*` as `Base*`, it may need to adjust the pointer for multiple and/or virtual base classes. The pointer caught by the core language, then, is a temporary. We can't return a `const Base**` pointing to that temporary adjusted pointer, because there's nowhere for the temporary adjusted pointer to live after the call to `try_cast` has returned.

In other words, the new design has a strict invariant: the pointer returned from `try_cast` always points to the in-flight exception object itself. It never points to any other object, such as a temporary or global. Thus, we must disallow:

```
using IntPtr = int*;
std::nullptr_t np;
auto p = std::make_exception_ptr(np);
  // The in-flight exception object is of type std::nullptr_t
const IntPtr *ex = old::try_cast<IntPtr>(p);
  // ex cannot possibly point to the in-flight exception object, because
try {
    std::rethrow_exception(p);
} catch (const IntPtr& ex) {
    // OK, ex refers to a temporary that lives only as long as this catch
}
```

Our solution is simply to extend our Mandates element to also forbid `std::try_cast` with a template argument of pointer or pointer-to-member type; these are the only two kinds of types where a core-language catch-handler parameter would sometimes bind to a temporary, so these are the only kinds of types we need to forbid. Later, we found that [ScyllaDB] had independently implemented the same solution (i.e. explicitly forbid pointer types) in 2022.

Throwing pointers is rare — probably unheard of in real code. This does prevent users from using `std::try_cast<const char*>(p)` to inspect the results of `throw "foo"`, which comes up sometimes in example code; but it shouldn't happen in real code.

Pattern matching

We expect that `try_cast` will be integrated in the pattern matching facility and will allow inspection of `exception_ptr` as follows:

```
inspect (eptr) {
   <logic_error> e => { ... }
   <exception> e => { ... }
   nullptr => { puts("no exception"); }
   __ => { puts("some other exception"); }
}
```

Other names considered but rejected

- `cast_exception_ptr<E>`
- `cast_exception<E>`
- `try_cast_exception_ptr<E>`
- `try_cast_exception<E>`
- `catch_as<E>`
- `exception_cast<E>`

Implementation

GCC, MSVC implementation is possible using available (but undocumented) APIs https://godbolt.org/z/ErePMf66h. Implementability was also confirmed by MSVC and libstdc++ implementors.

A similar facility is available in Folly and supports Windows, libstdc++, and libc++ on linux/apple/freebsd.

https://github.com/facebook/folly/blob/v2023.06.26.00/folly/lang/Exception.h
https://github.com/facebook/folly/blob/v2023.06.26.00/folly/lang/Exception.cpp

Implementation there under the names: folly::exception_ptr_get_object folly::exception_ptr_get_type

Extra constraint imposed by MSVC ABI: it doesn't have information stored to do a full dynamic_cast. It can only recover types for which a catch block could potentially match. This does not conflict with the `try_cast` facility offered in this paper.

Arthur has implemented P2927R1 `std::try_cast` in his fork of libc++; see [libc++] and [Godbolt].

Usage experience

A version of exception_ptr inspection facilities is deployed widely in Meta as a part of Folly's future continuation matching.

ScyllaDB implements almost exactly the wording of this proposal, under the name `try_catch<E>(p)`; see [ScyllaDB]. The only difference is that they return `E*` instead of `const E*`. We hear from them that they don't actually use the mutability for anything; and even if they did, they could add `const_cast` as mentioned above.

Proposed wording (relative to n4950)

In section [exception.syn] add definition for `try_cast` as follows:

```
exception_ptr current_exception() noexcept;
[[noreturn]] void rethrow_exception(exception_ptr p);
template <class E>
  const E* try_cast(const exception_ptr& p) noexcept;
template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

Modify paragraph 7 of section Exception propagation [propagation] as follows:

For purposes of determining the presence of a data race, operations on `exception_ptr` objects shall access and modify only the `exception_ptr` objects themselves and not the exceptions they refer to. Use of `rethrow_exception` or `try_cast` on `exception_ptr` objects that refer to the same exception object shall not introduce a data race.

Add the following paragraph immediately after paragraph 8 of section Exception propagation [propagation]:

template <class E>
const E* try_cast(const exception_ptr& p) noexcept;

Mandates: `E` is a *cv*-unqualified complete object type. `E` is not an array type. `E` is not a pointer or pointer-to-member type. *[Note: When `E` is a pointer or pointer-to-member type, a handler of type `const E&` can match without binding to the exception object itself. —end note]*

Returns: A pointer to the exception object referred to by `p`, if `p` is not null and a handler of type `const E&` would be a match [except.handle] for that exception object. Otherwise, `nullptr`.

Acknowledgments

Many thanks to those who provided valuable feedback, among them: Aaryaman Sagar, Barry Revzin, Gabriel Dos Reis, Jan Wilmans, Joshua Berne, Lee Howes, Lewis Baker, Michael Park, Peter Dimov, Ville Voutilainen, Yedidya Feldblum.

References

https://godbolt.org/z/E8n69xKjs (gcc and msvc implementation)

https://wg21.link/p0933 Runtime introspection of exception_ptr

https://wg21.link/p1066 How to catch an exception_ptr without even try-ing

https://wg21.link/p1371 Pattern Matching

https://github.com/scylladb/scylladb/blob/946d281/utils/exceptions.hh#L128-L151 ScyllaDb

An implementation of try cast and libc++ and matching godbolt:

https://github.com/Quuxplusone/llvm-project/commit/6e20a0b9d5a2280bfab8ab42bee841cfbcc4a8bd

https://godbolt.org/z/3Y8Gzfr7r